

Testcontainers

Visão geral da ferramenta

Funcionamento, recursos, ecossistema e avaliação técnica.

João Henrique



Dependências como código

O que é Testcontainers?

Uma família de bibliotecas para preparar dependências em contêineres durante testes.

O que gerencia

Criação, configuração, prontidão, endereços e encerramento dos serviços.

Como é utilizada

O código do teste solicita o ambiente e depois verifica o comportamento esperado.

Qual problema a ferramenta resolve?

Preparação manual

Instalar e configurar serviços antes de conseguir testar.

Estado compartilhado

Dados deixados por uma execução afetam a seguinte.

Ambientes diferentes

Versões, portas e disponibilidade variam entre máquinas.

Proposta: declarar e controlar as dependências no código dos testes.

Biblioteca, runtime e executor de testes

Executor de testes (JUnit 5)

Inicia os casos e avalia as asserções.

Biblioteca Testcontainers

Traduz a configuração em operações de contêiner.

Runtime com API Docker

Cria e executa os contêineres solicitados.

Serviços nos contêineres

Respondem aos clientes usados pelo sistema testado.

Posição na pirâmide de automação

Interface / ponta a ponta

Serviço / API / integração

Unidade: regras locais e feedback rápido

PAPEL DA FERRAMENTA

Preparar a infraestrutura necessária ao teste.

Uso típico: integração.
Também pode apoiar API
e testes com navegador.

Como funciona o ciclo de vida?

1

Declarar

Imagem e
configuração

2

Iniciar

Criação pelo
runtime

3

Esperar

Condições de
prontidão

4

Utilizar

Endereço e
operações

5

Limpar

Encerramento
e remoção

No exemplo: um campo `@Container` de instância para cada teste JUnit 5.

[Fonte: Testcontainers Java — integração JUnit 5](#)

Principais funcionalidades

Configuração como código

Imagem, variáveis, portas, redes e arquivos do ambiente de teste.

Espera por prontidão

Logs, healthchecks e HTTP, conforme a API e as features escolhidas.

Endereços dinâmicos

Consultar host e porta publicada ajuda a evitar colisões no host.

Inspeção e extensão

Comandos, logs, módulos prontos e configuração de imagens próprias.

[Fontes: Testcontainers Java — criação, espera, rede, comandos e arquivos](#)

Linguagens e módulos

Implementações

Java, Go, .NET, Node.js, Python, Rust e outras.

Cada linguagem possui sua própria API e documentação.

Módulos de serviços

Bancos, caches, brokers, emuladores e navegadores.

Configurações prontas para serviços específicos.

A cobertura de módulos e as versões variam entre implementações.

[Fontes: portal oficial e catálogo de módulos](#)

Integrações e tipos de teste

Executores

JUnit 5 executa os casos;
Maven prepara a suíte.

CI/CD

Runner com acesso ao runtime e aos
serviços criados.

Interface

Navegadores em contêiner com cliente
Selenium no Java.

Caixa-preta e branca

Infraestrutura para ambas; asserções e
cobertura são separadas.

[Fontes: integração JUnit 5, quickstart Java e módulo WebDriver](#)

Biblioteca, Desktop e Cloud

Bibliotecas

APIs incorporadas ao projeto para gerenciar dependências de teste.

Desktop

Aplicativo de apoio para inspecionar sessões, depurar serviços e selecionar runtimes.

Cloud

Serviço para executar dependências em contêineres remotos, no desenvolvimento e no CI.

Java e Docker local; Testcontainers Desktop registra a demonstração.

[Fonte: Testcontainers Desktop](#)

[Fonte: Testcontainers Cloud](#)

Requisitos e primeiros passos

Requisitos

JDK 21 e Maven.

Docker acessível pelo WSL.

Imagem PostgreSQL e recursos para executá-la.

Primeiros passos

Adicionar a biblioteca.

Escolher imagem ou módulo.

Criar o serviço, conectar o cliente e executar o teste.

No primeiro uso, o download da imagem pode dominar o tempo de preparo.

[Fonte: requisitos de runtime do Testcontainers](#)

Demonstração breve da ferramenta

```
@Container // inicia/remove por teste
final PostgreSQLContainer postgres =
    new PostgreSQLContainer(
        "postgres:16.4-alpine") // imagem
        .withCommand("postgres", "-c",
            "fsync=on"); // sincroniza disco
// @BeforeEach: host e porta dinâmicos
String host = postgres.getHost();
int porta = postgres.getMappedPort(5432);
```

Observe

Extensão prepara.

Porta dinâmica.

Remove após
cada teste.

```
python3 src/executar_dashboard.py # testes + dashboard
```

Maven direto: `mvn -B -o -f src/pom.xml test` · 3 testes; 0 falhas, erros e ignorados (02/10/2026).

Alternativas e diferenças

Opção	Abordagem	Ciclo de vida	Escolha típica
Testcontainers	Serviços em contêiner	API no teste	Dependências de teste
Docker Compose	Stack em YAML	Comandos / executor	Ambiente completo
H2 em memória	Banco na JVM	Conexões / aplicação	Testes de SQL
Mockito / mocks	Contrato simulado	Sem iniciar serviço	Testes unitários

Critérios: isolamento, integração à suíte e custo de execução.

Comparação qualitativa: H2 é outro banco; mocks simulam colaboradores. As opções podem coexistir.

Vantagens, custos e limites

Vantagens

Preparação explícita no teste.

Serviço real ou emulador escolhido.

Menos configuração manual e maior controle do ambiente.

Custos e limites

Inicialização, memória e rede.

Runtime e conectividade no CI.

Isolamento dos dados e limpeza exigem decisões da suíte.

O ambiente de teste reproduz as condições configuradas pelo projeto.

Adoção documentada

Uber - 2021

Uso com bancos em contêineres e portas dinâmicas para reduzir problemas de infraestrutura nos testes.

Capital One - 2020

Relato de testes portáteis com Spring Boot, Testcontainers e LocalStack.

Quarkus

Dev Services integra a ferramenta no preparo de dependências durante desenvolvimento e testes.

Fontes: relatos técnicos e documentação. Casos do ecossistema Java.

[Uber Engineering](#)

[Capital One Tech](#)

[Documentação Quarkus](#)

Conclusão: quando adotar a ferramenta?

Adotar quando o teste precisa de uma dependência controlada que execute comportamento relevante.

Boa adequação

Integrações com serviços e preparação automática do ambiente de teste.

Avaliar o custo

Regras locais rápidas e ambientes sem runtime disponível pedem outras opções.

Perguntas · Fontes completas no resumo da ferramenta e nas notas dos slides.

[Referências principais: testcontainers.com · documentação por linguagem · relatos de adoção.](#)